ML Experiment Number 10

Student Name :Tejas Shekhar Kamble

Batch :CS2

Roll Number :CS22

Aim

To implement DBSCAN clustering using Python for customer segmentation on the Mall Customer Segmentation dataset and identify arbitrary-shaped clusters along with spatial outliers (noise).

Objectives

1. To understand the principles of Density-Based Spatial Clustering of Applications with Noise (DBSCAN).
2. To implement DBSCAN clustering using Python.3. To preprocess and feature-scale tabular customer data using StandardScaler.
3. To determine optimal hyperparameters (ε and $min_samples$) using k-distance graph analysis.
4. To identify core points, border points, and spatial outliers (noise points denoted by label -1).
5. To evaluate clustering performance using metrics like Silhouette Score.

Relevant Theory

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN is an unsupervised non-parametric clustering algorithm that groups data points based on local density estimations. Unlike centroid-based algorithms (e.g., K-Means), DBSCAN can discover arbitrary-shaped clusters and automatically isolate noisy data points without requiring the user to specify the total number of clusters in advance.

Core Concepts and Parameters:

1. ε (Epsilon): The maximum radius of the neighborhood around a given data point x.
2. $min_samples$ (MinPts): The minimum number of points within the ε-neighborhood required to form a dense region.

Point ClassificationsCore Point:

- A point that has at least $min_samples$ within its ε-neighborhood.
- Border Point: A point that is within the ε-neighborhood of a Core Point but has fewer than $min_samples$ in its own neighborhood.
- Noise Point (Outlier): A point that is neither a Core Point nor a Border Point. Noise points are assigned a cluster label of -1.

Dataset Information

Get the Mall Customer Segmentation dataset from Kaggle using the following link: https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python

The dataset contains spending patterns and demographic attributes of mall customers:

- CustomerID: Unique identifier for each customer.
- Gender: Male or Female.
- Age: Age of the customer.
- Annual Income (k$): Annual income of the customer in thousands of dollars.
- Spending Score (1-100): Score assigned based on customer behavior and purchasing patterns.

For clustering, we will use: Annual Income (k$) and Spending Score (1-100).

Task 1: Import Libraries and Load Dataset

- Load essential libraries (numpy, pandas, matplotlib, seaborn, sklearn).

- Load the Mall Customer Segmentation dataset (Mall_Customers.csv).

In [ ]:

```
# Task 1: Import Libraries and Load Dataset

# Step 1: Import essential libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Step 2: Import Scikit-learn libraries
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import silhouette_score

# Step 3: Upload the Mall Customers dataset
from google.colab import files
import io

uploaded = files.upload()

# Step 4: Load the CSV file
filename = next(
    (name for name in uploaded if name.lower().endswith(".csv")),
    None
)

if filename is None:
    raise ValueError("Please upload a CSV file.")

df = pd.read_csv(io.BytesIO(uploaded[filename]))

# Step 5: Display the first five rows
print("First Five Rows of the Dataset:")
print(df.head())

# Step 6: Display dataset dimensions
print("\nDataset Shape:")
print(df.shape)

# Step 7: Display column names
print("\nColumn Names:")
print(df.columns.tolist())

# Step 8: Display dataset information
print("\nDataset Information:")
df.info()

# Step 9: Check missing values
print("\nMissing Values:")
print(df.isnull().sum())
```

```
# Step 10: Display statistical summary
print("\nStatistical Summary:")
print(df.describe())

# Step 11: Select the features required for DBSCAN
features = ["Annual Income (k$)", "Spending Score (1-100)"]
X = df[features].copy()

print("\nSelected Features:")
print(X.head())
```

Browse...

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Mall_Customers.csv to Mall_Customers.csv
First Five Rows of the Dataset:
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

Dataset Shape:
(200, 5)

Column Names:
['CustomerID', 'Gender', 'Age', 'Annual Income (k$)', 'Spending Score
(1-100)']

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB

Missing Values:
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64

Statistical Summary:
       CustomerID         Age  Annual Income (k$)  Spending Score (1-100)
count  200.000000  200.000000          200.000000              200.000000
mean   100.500000   38.850000           60.560000               50.200000
std     57.879185   13.969007           26.264721               25.823522
min      1.000000   18.000000           15.000000                1.000000
25%     50.750000   28.750000           41.500000               34.750000
50%    100.500000   36.000000           61.500000               50.000000
75%    150.250000   49.000000           78.000000               73.000000
max    200.000000   70.000000          137.000000               99.000000

Selected Features:
   Annual Income (k$)  Spending Score (1-100)
0                  15                      39
1                  15                      81
2                  16                       6
3                  16                      77
```

4 17 40

Task 2: Feature Selection and Standardization

- Select target features (Annual Income (k) and Spending Score (1-100)).
- Scale features using Standard Scaler to bring feature dimensions to a comparable range ($\mu=0, \sigma=1$), which is crucial for distance-based neighborhood computations.

In [ ]:
```
# Task 2: Feature Selection and Standardization

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# Step 1: Select the required features
features = ["Annual Income (k$)", "Spending Score (1-100)"]
X = df[features].copy()

print("Original Feature Data:")
print(X.head())

# Step 2: Check for missing values
print("\nMissing Values:")
print(X.isnull().sum())

# Handle missing values if present
X = X.fillna(X.median())

# Step 3: Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Convert scaled data into a DataFrame
X_scaled = pd.DataFrame(
    X_scaled,
    columns=features,
    index=X.index
)

# Step 4: Display standardized data
print("\nStandardized Feature Data:")
print(X_scaled.head().round(3))

# Step 5: Verify mean and standard deviation
print("\nMean after Standardization:")
print(X_scaled.mean().round(3))
```

```
print("\nStandard Deviation after Standardization:")
print(X_scaled.std(ddof=0).round(3))

print("\nTask 2 Completed Successfully!")
```

```
Original Feature Data:
   Annual Income (k$)  Spending Score (1-100)
0                  15                      39
1                  15                      81
2                  16                       6
3                  16                      77
4                  17                      40

Missing Values:
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64

Standardized Feature Data:
   Annual Income (k$)  Spending Score (1-100)
0              -1.739                  -0.435
1              -1.739                   1.196
2              -1.701                  -1.716
3              -1.701                   1.040
4              -1.663                  -0.396

Mean after Standardization:
Annual Income (k$)       -0.0
Spending Score (1-100)   -0.0
dtype: float64

Standard Deviation after Standardization:
Annual Income (k$)        1.0
Spending Score (1-100)    1.0
dtype: float64

Task 2 Completed Successfully!
```

Task 3: Hyperparameter Selection via k-Distance Graph

- Compute Nearest Neighbors using NearestNeighbors ($k = min_samples$).
- Plot sorted k-distances to locate the "elbow point," which indicates the optimal value for ε (Epsilon).

In [ ]:
```
# Task 3: Hyperparameter Selection via k-Distance Graph

import numpy as np
```

```python
import matplotlib.pyplot as plt
from sklearn.neighbors import NearestNeighbors

# Step 1: Set min_samples
min_samples = 5

# Step 2: Compute nearest neighbors
neighbors = NearestNeighbors(n_neighbors=min_samples)
neighbors.fit(X_scaled)

distances, indices = neighbors.kneighbors(X_scaled)

# Step 3: Extract the distance to the k-th nearest neighbor
k_distances = distances[:, min_samples - 1]

# Step 4: Sort distances in ascending order
k_distances = np.sort(k_distances)

# Step 5: Plot the k-distance graph
plt.figure(figsize=(12, 6), dpi=120)

plt.plot(
    range(1, len(k_distances) + 1),
    k_distances,
    color="blue",
    linewidth=2
)

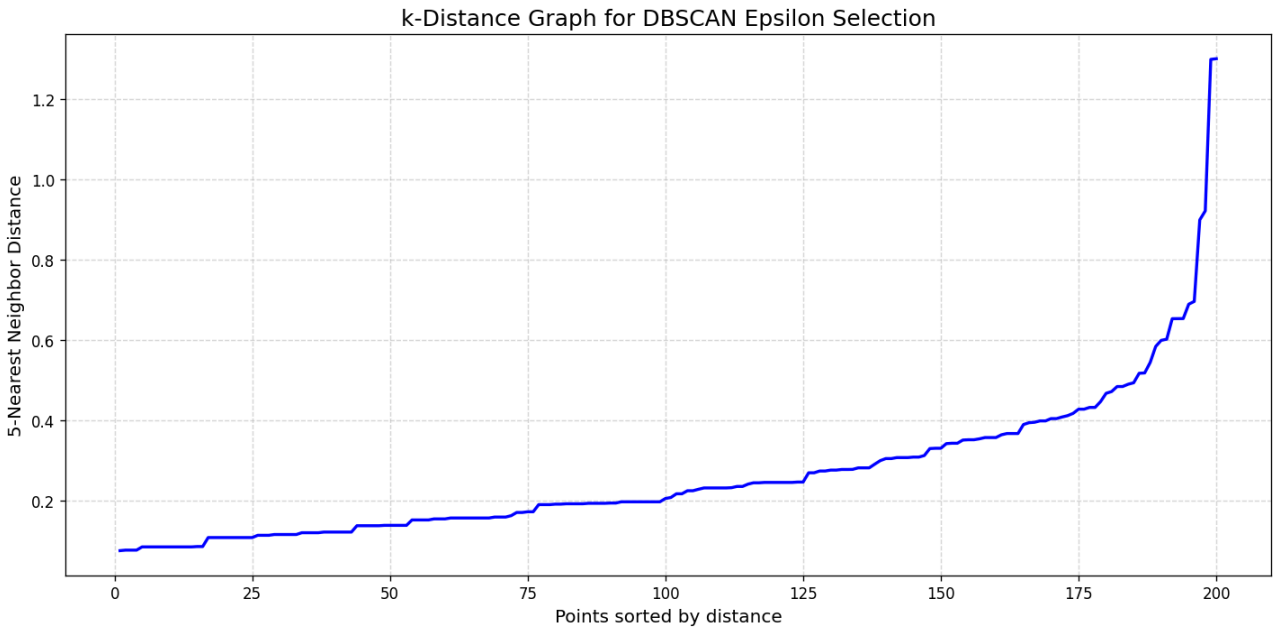
plt.xlabel("Points sorted by distance", fontsize=12)
plt.ylabel(f"{min_samples}-Nearest Neighbor Distance", fontsize=12)

plt.title(
    "k-Distance Graph for DBSCAN Epsilon Selection",
    fontsize=15
)

plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()

# Step 6: Display the largest distances to help inspect the elbow
print("Min Samples:", min_samples)
print("\nLargest 10 k-distances:")
print(np.round(k_distances[-10:], 3))

print("\nInspect the graph and choose epsilon near the elbow point.")
```




```
Min Samples: 5

Largest 10 k-distances:
[0.602 0.653 0.654 0.654 0.689 0.696 0.899 0.921 1.298 1.3  ]

Inspect the graph and choose epsilon near the elbow point.
```

Task 4: Implement DBSCAN Clustering

- Instantiate sklearn.cluster.DBSCAN with the identified ε and $min_samples$.
- Fit the model on scaled data to generate cluster assignments and noise labels (-1).

In [ ]:
```python
# Task 4: Implement DBSCAN Clustering

import numpy as np
import pandas as pd
from sklearn.cluster import DBSCAN

# Step 1: Set DBSCAN hyperparameters
# Replace eps with the value selected from your k-distance graph
eps_value = 0.4
min_samples = 5

# Step 2: Create the DBSCAN model
dbscan = DBSCAN(
    eps=eps_value,
    min_samples=min_samples
```

```
)

# Step 3: Fit the model and predict cluster labels
labels = dbscan.fit_predict(X_scaled)

# Step 4: Add cluster labels to the dataset
df["Cluster"] = labels

# Step 5: Display the first 10 cluster assignments
print("First 10 Customer Cluster Assignments:")
print(df[[
    "CustomerID",
    "Annual Income (k$)",
    "Spending Score (1-100)",
    "Cluster"
]].head(10).to_string(index=False))

# Step 6: Count customers in each cluster
print("\nNumber of Customers in Each Cluster:")
print(df["Cluster"].value_counts().sort_index())

# Step 7: Count clusters and noise points
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise = np.sum(labels == -1)

print("\nDBSCAN Results")
print("Epsilon (eps):", eps_value)
print("Minimum Samples:", min_samples)
print("Number of Clusters:", n_clusters)
print("Number of Noise Points:", n_noise)
```

```
First 10 Customer Cluster Assignments:
 CustomerID  Annual Income (k$)  Spending Score (1-100)  Cluster
          1                  15                      39        0
          2                  15                      81        0
          3                  16                       6        1
          4                  16                      77        0
          5                  17                      40        0
          6                  17                      76        0
          7                  18                       6        1
          8                  18                      94       -1
          9                  19                       3        1
         10                  19                      72        0

Number of Customers in Each Cluster:
Cluster
-1     15
 0    115
 1     11
 2     32
 3     27
Name: count, dtype: int64

DBSCAN Results
Epsilon (eps): 0.4
Minimum Samples: 5
Number of Clusters: 4
Number of Noise Points: 15
```

Task 5: Evaluate DBSCAN Clustering

- Cluster Count: Calculate the number of clusters generated (excluding noise points).
- Noise Point Analysis: Count and evaluate identified spatial outliers (label -1).
- Silhouette Score: Calculate Silhouette Score to evaluate cluster cohesion and separation.

In [ ]:
```python
# Task 5: Evaluate DBSCAN Clustering

import numpy as np
from sklearn.metrics import silhouette_score

# Step 1: Calculate the number of clusters, excluding noise (-1)
unique_labels = set(labels)

n_clusters = len(unique_labels - {-1})

print("Number of Clusters:", n_clusters)

# Step 2: Count noise points
```

```python
n_noise = np.sum(labels == -1)
total_points = len(labels)

print("Number of Noise Points:", n_noise)
print("Total Data Points:", total_points)

# Step 3: Calculate noise percentage
noise_percentage = (n_noise / total_points) * 100

print("Noise Percentage: {:.2f}%".format(noise_percentage))

# Step 4: Calculate the number of points in each cluster
print("\nCluster Distribution:")

for cluster_id in sorted(unique_labels):
    count = np.sum(labels == cluster_id)

    if cluster_id == -1:
        print(f"Noise (-1): {count} points")
    else:
        print(f"Cluster {cluster_id}: {count} points")

# Step 5: Calculate Silhouette Score
# Exclude noise points from the score calculation
non_noise_mask = labels != -1

X_clustered = X_scaled[non_noise_mask]
labels_clustered = labels[non_noise_mask]

n_valid_clusters = len(np.unique(labels_clustered))

if 2 <= n_valid_clusters < len(labels_clustered):
    score = silhouette_score(
        X_clustered,
        labels_clustered
    )
    print("\nSilhouette Score: {:.4f}".format(score))
else:
    print("\nSilhouette Score cannot be calculated.")
    print("At least 2 non-noise clusters are required.")

# Step 6: Display evaluation summary
print("\n========== DBSCAN EVALUATION ==========")
print("Clusters (excluding noise):", n_clusters)
print("Noise points:", n_noise)
print("Noise percentage: {:.2f}%".format(noise_percentage))

print("\nTask 5 Completed Successfully!")
```

```
Number of Clusters: 4
Number of Noise Points: 15
Total Data Points: 200
Noise Percentage: 7.50%

Cluster Distribution:
Noise (-1): 15 points
Cluster 0: 115 points
Cluster 1: 11 points
Cluster 2: 32 points
Cluster 3: 27 points

Silhouette Score: 0.4781

========== DBSCAN EVALUATION ==========
Clusters (excluding noise): 4
Noise points: 15
Noise percentage: 7.50%

Task 5 Completed Successfully!
```

Task 6: Analyze Results and Cluster Statistics

- Extract cluster labels and append them back to the original dataset.
- Calculate mean statistics (e.g., average Annual Income and Spending Score) per cluster to interpret final customer segments and evaluate detected noise points.

In [ ]:
```python
# Task 6: Analyze Results and Cluster Statistics

import pandas as pd
import numpy as np

# Step 1: Append DBSCAN cluster labels to the original dataset
df["Cluster"] = labels

print("Dataset with Cluster Labels:")
print(df.head(10).to_string(index=False))

# Step 2: Calculate statistics for each cluster
# Exclude noise (-1) from normal customer segment statistics
cluster_data = df[df["Cluster"] != -1]

cluster_stats = cluster_data.groupby("Cluster").agg(
    Customer_Count=("CustomerID", "count"),
    Average_Age=("Age", "mean"),
    Average_Annual_Income=("Annual Income (k$)", "mean"),
```

```
    Average_Spending_Score=("Spending Score (1-100)", "mean")
).round(2)

print("\n========== CLUSTER STATISTICS ==========")
print(cluster_stats)

# Step 3: Analyze noise points
noise_data = df[df["Cluster"] == -1]

print("\n========== NOISE POINT ANALYSIS ==========")
print("Total Noise Points:", len(noise_data))

if not noise_data.empty:
    print("\nNoise Point Details:")
    print(
        noise_data[
            ["CustomerID", "Age", "Annual Income (k$)",
             "Spending Score (1-100)", "Cluster"]
        ].to_string(index=False)
    )

    print("\nAverage Statistics of Noise Points:")
    print(
        noise_data[
            ["Age", "Annual Income (k$)", "Spending Score (1-100)"]
        ].mean().round(2)
    )
else:
    print("No noise points detected.")

# Step 4: Interpret customer segments
print("\n========== CUSTOMER SEGMENT INTERPRETATION ==========")

for cluster_id, row in cluster_stats.iterrows():
    income = row["Average_Annual_Income"]
    spending = row["Average_Spending_Score"]

    if income >= 60 and spending >= 60:
        segment = "High Income - High Spending"
    elif income >= 60 and spending < 40:
        segment = "High Income - Low Spending"
    elif income < 40 and spending >= 60:
        segment = "Low Income - High Spending"
    elif income < 40 and spending < 40:
        segment = "Low Income - Low Spending"
    else:
        segment = "Moderate Income/Spending"

    print(f"Cluster {cluster_id}: {segment}")

print("\nTask 6 Completed Successfully!")
```

Dataset with Cluster Labels:

CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
1	Male	19	15	39	0
2	Male	21	15	81	0
3	Female	20	16	6	1
4	Female	23	16	77	0
5	Female	31	17	40	0
6	Female	22	17	76	0
7	Female	35	18	6	1
8	Female	23	18	94	-1
9	Male	64	19	3	1
10	Female	30	19	72	0

========== CLUSTER STATISTICS ==========

Cluster	Customer_Count	Average_Age	Average_Annual_Income	Average_Spending_Score
0	115	39.54	48.30	51.73
1	11	49.36	23.73	8.91
2	32	32.75	80.88	83.62
3	27	41.26	83.93	14.44

========== NOISE POINT ANALYSIS ==========

Total Noise Points: 15

Noise Point Details:

CustomerID	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
8	23	18	94	-1
12	35	19	99	-1
20	35	23	98	-1
170	32	87	63	-1
185	41	99	39	-1
188	28	101	68	-1
192	32	103	69	-1
193	33	113	8	-1
194	38	113	91	-1
195	47	120	16	-1
196	35	120	79	-1

```
        197   45                  126                     28       -1
        198   32                  126                     74       -1
        199   32                  137                     18       -1
        200   30                  137                     83       -1

Average Statistics of Noise Points:
Age                        34.53
Annual Income (k$)         96.13
Spending Score (1-100)     61.80
dtype: float64

========== CUSTOMER SEGMENT INTERPRETATION ==========
Cluster 0: Moderate Income/Spending
Cluster 1: Low Income - Low Spending
Cluster 2: High Income - High Spending
Cluster 3: High Income - Low Spending

Task 6 Completed Successfully!
```

Conclusion

The experiment successfully implemented DBSCAN clustering on the Mall Customer Segmentation dataset. Feature standardization ensured balanced neighborhood distance calculations across income and spending metrics. By analyzing k-distance graphs, optimal ε and $min_samples$ parameters were selected, enabling the discovery of density-defined customer segments and explicit identification of spatial noise points (outliers) without requiring a pre-defined cluster count.